PowerShell + DevOps
Global Summit

POWERSHELL DEVOPS 2026 GLOBAL SUMMIT

LOST IN TRANSLATION

THE MANY LANGUAGE MODES OF POWERSHELL

Josh Corrick

Engineer @ Microsoft

April 13-16, 2026

CC BY SA

Jane Street

fleet

Chocolatey

PATCH MY PC

PDQ

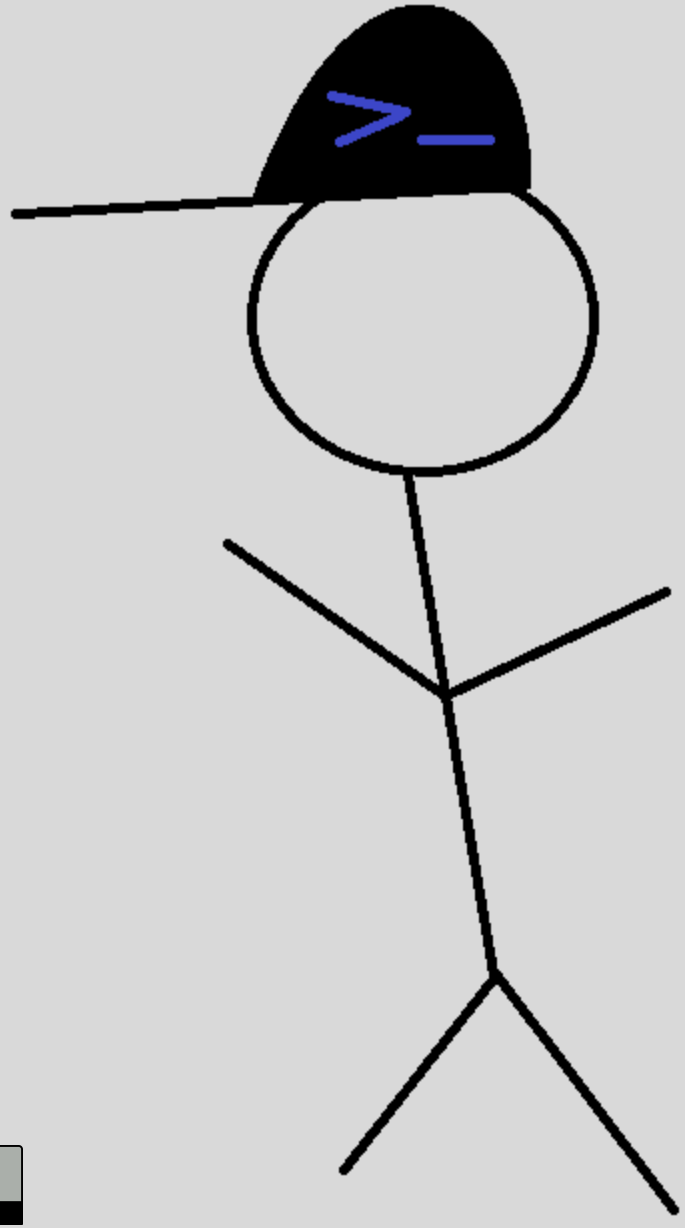
SynEdgy

CC BY SA

About Me

- Engineer @ Microsoft
- I write tools for my team
- Love helping others succeed

CC BY SA



CC BY SA

Your PowerShell Does Not Work

```
Cannot invoke method. Method invocation is supported only on core types in this language mode.
At line:1 char:1
+ $class.TestNonCoreType()
+ ~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : MethodInvocationNotSupportedInConstrainedLanguage
```

CC BY SA

Where Do I Start?

- What are Language modes?
- Why do they exist?
- When do these errors happen?
- How do I prevent this from happening?

CC BY SA

Language Modes

- Full Language (FLM)
- Constrained (CLM)
- Restricted
- NoLanguage

```
Get-Help about_language_modes
```

CC BY SA

Full Language

- Everything is allowed (cmdlets, scriptblocks, .net, etc.)
- Default language mode on all versions
- This is the one we all Love

CC BY SA

Full Language

```
[FLM]> Add-Type -AssemblyName System.IO.Compression.FileSystem
[FLM]> [System.IO.Compression.ZipFile]::ExtractToDirectory($zipPath, $destinationPath)
```

```
[FLM]> $data = [PSCustomObject]@{ I_have='Something'; I_know='Something'; I_am='Something' }
```

```
[FLM]> [math]::Round([math]::pi, 9)
```

CC BY SA

Constrained

- Basic language elements
- Control flow still works
- Type usage is restricted

CC BY SA

Constrained

```
[CLM]> Add-Type -AssemblyName System.IO.Compression.FileSystem
Add-Type : Cannot add type. Definition of new types is not supported in this language mode.
```

```
[CLM]> $data = [PSCustomObject]@{ I_have='Something'; I_know='Something'; I_am='Something' }
Cannot convert value to type "System.Management.Automation.LanguagePrimitives+InternalPSCustomObject".
Only core types are supported in this language mode.
```

```
[CLM]> [math]::Round([math]::pi,9)
Cannot invoke method. Method invocation is supported only on core types in this language mode.
```

CC BY SA

Restricted

```
[Restricted] > $test = 'this is a var'
Assignment statements are not allowed in restricted language mode or a Data Section.
```

NoLanguage

```
PS> $test
The syntax is not supported by this runspace. This can occur if the runspace is in no-language mode.
```

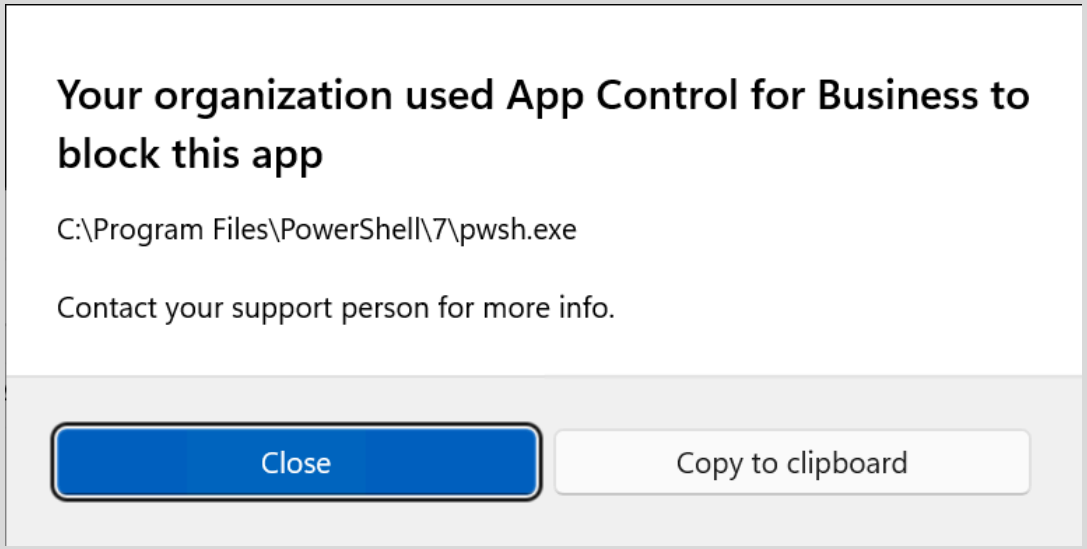
CC BY SA

Why do Language Modes exist?

CC BY SA

Application Control

ACfB / WDAC / UMCI / DeviceGuard



- Windows security feature
- Applies to **all** users
- Applies to **all** applications
- Untrusted PowerShell in CLM
- Trusted PowerShell in FLM

CC BY SA

Cool, but how do we check if we are in CLM?

```
[Constrained Language Mode] #PS 7+
$Host.Runspace.LanguageMode
$ExecutionContext.SessionState.LanguageMode
```

```
# Windows PowerShell only
Get-AppLockerPolicy -Effective | Select-Object -ExpandProperty RuleCollections
Get-ComputerInfo -Property *CodeIntegrity*
```

CC BY SA

CLM Gallery of Errors

CC BY SA

Method Calls on Untrusted Types

```
Cannot invoke method. Method invocation is supported only on core types in this language mode.
At line:1 char:1
+ $class.TestNonCoreType()
+ ~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : MethodInvocationNotSupportedInConstrainedLanguage
```

CC BY SA

Method Calls on Untrusted Types

Allowed Types

String, Int, Bool, DateTime, Guid, Hashtables, PSCustomObject, uri, X509Certificate, xml ...

```
# For the full list
Get-Help about_language_modes
```

CC BY SA

Solution: Method Calls on Untrusted Types

- Call only methods on allowed types
- Call non-core methods in signed code
- Use Built-In Cmdlets and alternative patterns

CC BY SA

Example: Method Calls on Untrusted Types

```
[FLM]> $sb = New-Object System.Text.StringBuilder
[FLM]> $null = $sb.AppendJoin(' ',$Words)
[FLM]> $sb.ToString()
```

```
[CLM]> $text = ''
[CLM]> $text += 'A '
[CLM]> $text += 'Test.'
```

CC BY SA

Dot-Sourcing Across Trust Boundaries

```
. : The term 'C:\Users\user\Documents\WindowsPowerShell\Modules\MySignedModule\Private.ps1' is not recognized as the
name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was
included, verify that the path is correct and try again.
At C:\Users\user\Documents\WindowsPowerShell\Modules\MySignedModule\MySignedModule.psm1:1 char:3
+ . $PSScriptRoot\Private.ps1
+   ~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (C:\Users\user\D...ule\Private.ps1:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

CC BY SA

Dot-Sourcing Across Trust Boundaries

```
VERBOSE: Loading module from path 'C:\Users\user\Documents\WindowsPowerShell\Modules\MyOddModule\MyOddModule.psd1'.
Import-Module : Importing *.ps1 files as modules is not allowed in ConstrainedLanguage mode.
At line:1 char:1
+ Import-Module MyOddModule -Verbose -Force
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (:) [Import-Module], InvalidOperationException
    + FullyQualifiedErrorId : Modules_ImportPSFileNotAllowedInConstrainedLanguage,Microsoft.PowerShell.Commands.Import
   ModuleCommand
```

CC BY SA

Solution: Dot-Sourcing Across Trust Boundaries

- Do not dot-source untrusted scripts
- Prefer Import-Module and `.psm1`
- Don't use `.ps1` in `.psd1` files
- Sign files if they are going to be dot-sourced

CC BY SA

Example: Dot-Sourcing Across Trust Boundaries

```
. $PSScriptRoot\Private.ps1 # NOT in CLM
Import-Module .\MyModule.psm1 # OK in CLM
```

```
@{
    RootModule = 'MyModule.psm1'  # OK
    NestedModules = @('NestedModule.psm1','NestedModule2.psm1') # OK
    ScriptsToProcess = @('init.ps1')   # NOT in CLM
}
```

CC BY SA

Loading Non-Core Types

```
New-Object : Cannot create type. Only core types are supported in this language mode.
At C:\Users\user\Documents\WindowsPowerShell\Modules\mymodule\MyModule.psm1:35 char:14
+     $shell = New-Object -ComObject WScript.Shell
+              ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (:) [New-Object], PSNotSupportedException
    + FullyQualifiedErrorId : CannotCreateComTypeConstrainedLanguage,Microsoft.PowerShell.Commands.NewObjectCommand

You cannot call a method on a null-valued expression.
At C:\Users\user\Documents\WindowsPowerShell\Modules\mymodule\MyModule.psm1:36 char:5
+     $shell.PopUp($Message)
+     ~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : InvokeMethodOnNull
```

CC BY SA

Solution: Loading Non-Core Types

- Only use core types
- Lint code for non-core types
- Call non-core types in signed code

CC BY SA

Example: Loading Non-Core Types

```
# NOT ALLOWED in CLM
$content = New-Object System.Net.WebClient
$xml = New-Object -ComObject MSXML2.DOMDocument
```

```
# Use core types and cmdlets instead
$content = Invoke-RestMethod ...
$xmlContent = [System.Xml.XmlDocument]::new()
```

CC BY SA

Defining New Types

```
Add-Type : Cannot add type. Definition of new types is not supported in this language mode.
At C:\Users\user\Documents\WindowsPowerShell\Modules\mymodule\MyModule.psm1:45 char:5
+     Add-Type @"
+     ~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (:) [Add-Type], PSNotSupportedException
    + FullyQualifiedErrorId : CannotDefineNewType,Microsoft.PowerShell.Commands.AddTypeCommand

Unable to find type [Win32].
At C:\Users\user\Documents\WindowsPowerShell\Modules\mymodule\MyModule.psm1:55 char:1
+ [Win32]::MessageBox(0, "$Text", "$Title", 0)
+ ~~~~~~~
```

CC BY SA

Solution: Defining New Types

- Custom type definitions are blocked in CLM
- Lint for add-type calls
- Keep type creation in signed code
- This goes for using `class` keyword also

CC BY SA

Example: Defining New Types

```
# NOT ALLOWED in CLM
class MyClass { [string]$Name }
Add-Type @"
public class Win32 { }
"@
```

```
# Use hashtable or PSCustomObject instead
$obj = @{ Name = 'Test' }
```

CC BY SA

Type Accelerators in CLM

```
Cannot convert value to type "System.Management.Automation.LanguagePrimitives+InternalPSCustomObject". Only core types
are supported in this language mode.
At C:\Users\user\Documents\WindowsPowerShell\Modules\mymodule\MyModule.psm1:23 char:5
+     [PSCustomObject]@{
+     ~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [], RuntimeException
    + FullyQualifiedErrorId : ConversionSupportedOnlyToCoreTypes
```

CC BY SA

Solution: Type Accelerators in CLM

- Type casts fail by design
- Prefer explicit object construction
- Lint for `[PSCustomObject]` creation
- Leverage `New-Object` and `Add-Member`

CC BY SA

Example: Type Accelerators in CLM

```
#Blocked in CLM
  [PSCustomObject]@{
      LanguageMode = $LanguageMode
      Math = $math
      Phrase = $phrase
  }
```

```
  $obj = New-Object -TypeName PSObject
  Add-Member -InputObject $obj -MemberType NoteProperty -Name "LanguageMode" -Value $LanguageMode
  Add-Member -InputObject $obj -MemberType NoteProperty -Name "Math" -Value $Math
  Add-Member -InputObject $obj -MemberType NoteProperty -Name "Phrase" -Value $Phrase
  $obj
```

CC BY SA

Runtime Type Conversion

```
PS C:\Users\user\Desktop> $obj = New-Object PSObject -Property @{
>>     Name = "Test"
>>     Value = 42
>> }
PS C:\Users\user\Desktop> Test-NonCoreObject -Hashtable $obj
Test-NonCoreObject : Cannot process argument transformation on parameter 'Hashtable'. Cannot convert value "@{Value=42;
Name=Test}" to type "System.Collections.Hashtable". Error: "Cannot convert hashtable to an object of the following type:
System.Collections.Hashtable. Hashtable-to-Object conversion is not supported in constrained language mode, restricted language
mode or a Data section."
At line:1 char:31
+ Test-NonCoreObject -Hashtable $obj
+                               ~~~~
    + CategoryInfo          : InvalidData: (:) [Test-NonCoreObject], ParameterBindingArgumentTransformationException
    + FullyQualifiedErrorId : ParameterArgumentTransformationError,Test-NonCoreObject
```

CC BY SA

Solution: Runtime Type Conversion

- Lint for type conversions
- Test cmdlets for these errors
- Validate first, then convert in signed code
- Construct your type in signed code

CC BY SA

Example: Runtime Type Conversion

```
$obj = New-Object PSObject -Property @{
    Name = "Test"
    Value = 42
}
...
function Test-NonCoreObject {
    [CmdletBinding()]
    param (
        [Hashtable]$Hashtable
....
#Blocked in CLM even if signed
Test-NonCoreObject -Hashtable $obj
```

CC BY SA

Example: Runtime Type Conversion

```
function Test-NonCoreObject {
    [CmdletBinding()]
    param (
        # provide as a Hashtable
        [Hashtable]$Hashtable,
        # or take as a Custom Object and convert later
        [PSCustomObject]$CustomObject
```

CC BY SA

Tools

CC BY SA

Use AI

- Create `AGENTS.md`, `SKILL.md`, `.instructions.md`, `.agent.md`
- Define a plugin marketplace for Claude
- Leverage the PowerShell MCP

CC BY SA

Use AI

```
## Agent Roles

- **ScriptGenAgent**: Generates PowerShell scripts compatible with CLM restrictions.
When generating code, always validate output against the CLM checklist (see below)
before presenting it. Never produce code that uses disallowed features.
- **CLMReviewAgent**: Reviews scripts for compliance with CLM, highlighting
disallowed features or risky patterns. Always read the full contents of every file
in a module before making compliance judgments — do not guess based on file names alone.
```

CC BY SA

Linting

- PSScriptAnalyzer version 1.25.0+
  - PSUseConstrainedLanguageMode
- CodeQL for binary modules only
  - No official PowerShell support, but checkout GitHub issue 17927

CC BY SA

Linting

```
$settings = @{
      Rules = @{
          PSUseConstrainedLanguageMode = @{
              Enable = $true
              IgnoreSignatures = $true
          }
      }
}
Invoke-ScriptAnalyzer -Path .\MyOddModule\ -Settings $settings -IncludeRule PSUseConstrainedLanguageMode
```

CC BY SA

Testing in CLM

- Write Tests in Pester specifically for CLM
- Make build nodes specifically for CLM tests
- Manually test with ACfB/WDAC enforced CLM
- Validate behavior and collect failures via telemetry

CC BY SA

Testing in CLM

```
...
    run: |
        # Set Constrained Language Mode
        $ExecutionContext.SessionState.LanguageMode = "ConstrainedLanguage"
        Write-Host "Language Mode: $($ExecutionContext.SessionState.LanguageMode)"

        # Test 1: Add-Type (should fail in CLM)
        Write-Host "`n--- Test 1: Add-Type ---"
        try {
        Add-Type -TypeDefinition 'public class Demo { public static string SayHi() { return "hi"; } }'
        Write-Host "PASS (unexpected): Add-Type succeeded"
        } catch {
        Write-Host "EXPECTED: Add-Type blocked: $_"
        }
```

CC BY SA

Testing in CLM

clm-languagemode

succeeded now in 15s

- Set up job
- Checkout repo
- Run CLM Test (ConstrainedLanguage via LanguageMode)

```
1   ▶ Run # Set Constrained Language Mode
65  Language Mode: ConstrainedLanguage
66
67  --- Test 1: Add-Type ---
68  EXPECTED: Add-Type blocked: Cannot add type. Definition of new types is not supported in this language mode.
69
70  --- Test 2: Allowed cmdlets ---
71  PASS: Get-Process succeeded - AzureArcSysTray
72
```

CC BY SA

Testing in CLM

```
...
function Invoke-InConstrainedRunspace {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Script,

        [hashtable]$Parameters
    )

    $initialSessionState = [System.Management.Automation.Runspaces.InitialSessionState]::CreateDefault()
    $initialSessionState.LanguageMode = [System.Management.Automation.PSLanguageMode]::ConstrainedLanguage

    $runspace = [System.Management.Automation.Runspaces.RunspaceFactory]::CreateRunspace($initialSessionState)
    $runspace.Open()
...
```

CC BY SA

Signing Code

- Public code signing certs
- Azure Artifact Signing (Trusted Signing)
- Enterprise PKI
- Use a Hardware Security Module (HSM)

CC BY SA

Signing Code

```
$cert = Get-PfxCertificate .\PSHSummit-CodeSign.pfx
Set-AuthenticodeSignature -FilePath .\MyModule.psm1 -Certificate $cert
```

```
signtool.exe sign /f CodeSign.cer /csp "Hardware Security Module"
/kc KeyContainerName .\MyModule.psm1
```

CC BY SA

Summary

- Code for least privilege first
- Use AI to accelerate authoring
- Lint for CLM anti-patterns
- Test in enforced CLM
- Sign and trust your code

CC BY SA

THANK YOU

FEEDBACK IS A GIFT

Please review this session via the mobile app

Questions? Find me in after the talk

April 13-16, 2026

CC BY SA

Appendix

Helpful Links:

- PowerShell Constrained Language Mode
- PowerShell Module Function Export in Constrained Language
- PowerShell Constrained Language Mode and the Dot-Source Operator
- PowerShell 7 Security - Anam Navied - PSConfEU 2025
- dbatools is moving to Azure Trusted Signing

CC BY SA